

Name: _____ Score: _____

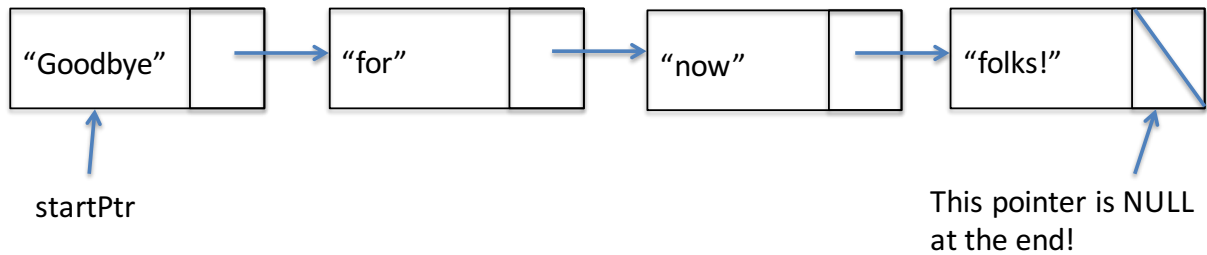
ECE 2036 Final Exam

Instructions

1. You may have **THREE** 8 ½ x 11 or A4 sheets of paper with **HANDWRITTEN** information on the front and back to use during the test.
2. You do not need a calculator.
3. You must turn off your cell phone. During the last hour of the test, the proctor to the test will provide the time in 10-minute increments. For the last 10 minutes of the test, the proctor will provide a 5 minute warning, 2 minute warning, and 1 minute warning for the end of the testing period.
4. If you need to go to the restroom, you must ask the proctor for permission AND leave your cell phone with the proctor of the exam.
5. When you are done with the test, make sure your name is on the front page. **YOU MAY KEEP YOUR AID SHEETS; HOWEVER, DO NOT WRITE DOWN FINAL PROBLEMS ON YOUR SHEET.**

Problem 1 (15 points): Linked Lists

I would like for you to use the class definition of Node to make the following data structure in the main code on the next page. Please include appropriate code as indicated by comments.



```
#include <iostream>
using namespace std;
```

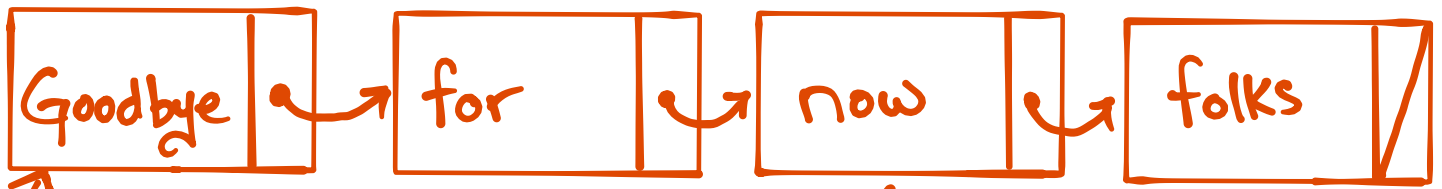
```
class Node
{
public:
    Node(string input):word(input),nextPtr(NULL) {}

    string getWord() const {return(word);}
    Node * getNextPtr() const {return(nextPtr);}
    void setWord(const string & input) {word = input;}
    void setNextPtr(Node * input) {nextPtr = input;}
```

```
private: public:
    string word;
    Node * nextPtr;
```

```
};
```

I forgot that I changed
to public during test for
simplicity.



```
int main()
```

```
{
    Node * startPtr = NULL; // Use this pointer to keep track of the start of your linked list.
    Node * currPtr = NULL; // Use this as a pointer to help manipulate and build data structure.
```

```
// 1. Insert code here to build the data structure with
// data as indicated on the previous page.
```

```
startPtr = new Node ("Good bye");
currPtr = startPtr;
currPtr->nextPtr = new Node ("for");
currPtr = currPtr->nextPtr;
currPtr->nextPtr = new Node ("now");
currPtr = currPtr->nextPtr;
currPtr->nextPtr = new Node ("folks!");
```

```
// 2. Go through and print the data for everything in the linked list. I will start it for you!
```

```
currPtr = startPtr;
while (currPtr != NULL)
{
    cout << currPtr->word << " ";
    currPtr = currPtr->nextPtr;
}
cout << endl;
```

```
// 3. Now delete the data so that there are no memory leaks!
```

```
currPtr = startPtr;
while (currPtr != NULL)
{
    curr = curr->nextPtr;
    delete startPtr;
    startPtr = currPtr;
}
```

```
}
```

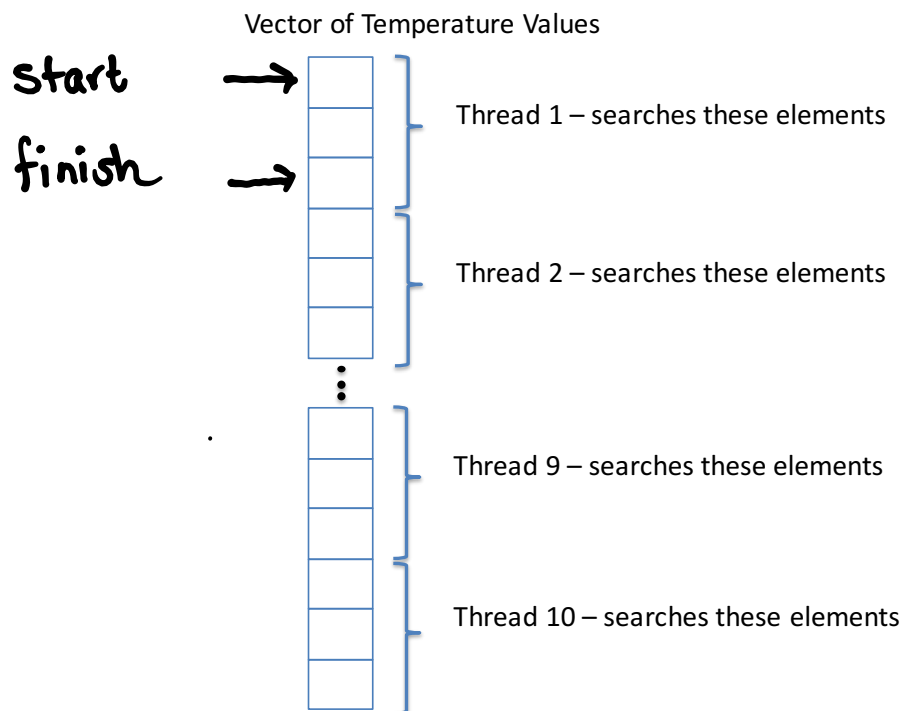
← move curr pointer to next element
 ← it is okay to lose the startPtr because we have the needed information in currPtr.

Problem 2 (10 points): Multi-Threaded Programming

Assume that NASA satellites over the last twenty years have recorded temperature reading for locations over the entire planet. I would like to determine the number of instances that the temperature reading is greater than 100 degrees Fahrenheit. To do this, I will read these temperature values in from a text file and store them in a vector of doubles. To find these values, this I will launch 10 different threads to search different parts of the vector quickly (as seen in the figure below) for all instances where the temperature is greater than or equal to 100 degrees Fahrenheit. Please note that the text file with the temperatures is simply a list of unordered real numbers separated by white space.

Please add to the code on the following pages according to the comments in **bold**. Please use the commands associated with the ~~gthreads~~ library discussed in class and lab.

c++ threads



* Note that I am changing this problem to use with c++ standard NOT gthreads.

```
#include <iostream>
#include <fstream>
#include <gthread.h> #include <thread>
#include <stdlib.h>
#include <vector>
```

```
using namespace std;
```

// 1. Here you will need to define mutex object(s) according to the gThreads library
// discussed in class.

mutex numMutex;

```
int numTimes;
vector<double> TemperatureReadings; // Assume this is filled
```

```
int readInTemperatureFromFile();
void searchArray(int start, int finish);
```

```
int main()
{
    int numPoints;
    int delta;
    int start;
    int finish;
    thread t[10]; // array of thread objects
    numPoints = readInTemperatureFromFile();
    cout << "The number of points in the file are: " << numPoints << endl;
```

```
    if (numPoints > 10)
    {
        delta = numPoints/10+1; // This +1 is needed to make sure you go through the entire list

        start = 0;
        for (int threadNumber=1; threadNumber <= 10; threadNumber++)
        {
            finish = start + delta;

            if (finish > numPoints)
            { finish = (numPoints-1); }
```

// 2. Here you will need to launch the thread ~~here you will need to launch the thread~~

t[threadNumber-1] = thread(searchArray, start, finish);

```
        start += (delta+1);
    }
}
else
{
    cerr << "There are not enough points in your temp file" << endl;
}
```

// 3. Is there something missing here? If so, add it; otherwise, move on!

for (int i=0; i < 10; i++) { t[i].join(); }

```
cout << "The temp goes above 100 : " << numTimes << endl;
```

```
} //end
```

**//4. Please create code for this THREAD that sequentially searches the
// TemperatureReadings vector from start to finish looking for temperatures
//that are greater than or equal to 100 degrees Fahrenheit**

```
void searchArray(int start, int finish)
{
```

```
    int tempCounter = 0;
    for (int i = start; i <= finish; i++)
    {
        if ( TemperatureReadings[i] > 100)
            tempCounter++;
    }

    num Mutex.lock();
    numTimes += tempCounter;
    num Mutex.unlock();
```

```
}
```

```
//-----
int readInTemperatureFromFile()
{
    ifstream inputFile("tempFile.txt",ios::in);
    double tempDouble;
    int counter=0;

    while (inputFile >> tempDouble)
    {
        //push the value on the back of the vector
        TemperatureReadings.push_back(tempDouble);
        counter++;
    }

    inputFile.close();
    return(counter);
}
```

Problem 3 (15 points): Letter Histograms

The goal of this problem is to complete a program that reads in a text file and creates a histogram of the number of occurrences of each letter. FOR SIMPLICITY YOU CAN ASSUME THAT ALL CHARACTERS ARE LOWER CASE. You can also ignore punctuation marks. I would like for you to make a vector of letter objects that you call CharUnit. This object will have a particular letter associated with it and the frequency that it appears in the file. Fill in the program on the following pages according to the comments in bold.

**//1. There are some missing include files. Please add missing
// ones that are needed so that this code will compile.**

```
#include <iostream>
#include <string>
#include <fstream>
using namespace std;
```

#include <vector>

```
class CharUnit
```

```
{
```

//2. Assume that you have no set and get functions. You will need to do

//something special here to get printVector and makeLetterHistogram to work.

```
friend void printVector ( vector <CharUnit> & );
friend void makeLetterHistogram ( vector <CharUnit> & );
```

```
public:
```

```
CharUnit(char c, int f): character(c), frequency(f) {}
```

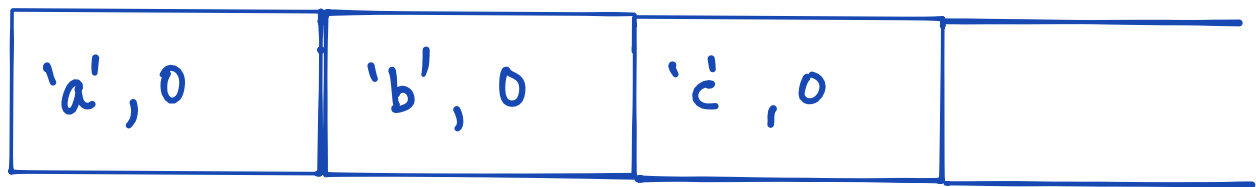
```
private:
```

```
char character;
```

```
int frequency;
```

```
};
```

```
vector <CharUnit> charVector;
```



charUnit

```
int main()
```

```
{
```

```
//3. First initialize the vector charVector with the letters of the alphabet
```

```
// (lower case ONLY!). Assume that the initial frequency is zero.
```

```
// Please put the 26 CharUnit's in the vector in alphabetical order according
```

```
// to the character data member. To get full credit, be as efficient as possible.
```

```
for (int i=0 ; i < 26; i++)
    charVector.push_back (CharUnit('a'+i, 0));
```

```
makeLetterHistogram(charVector);
```

```
printVector(charVector);
```

```
}//end of main
```

```
// 4. Now show the implementation of the global function printVector, which is
```

```
// called in the main function. Make sure you print the character and the frequency
```

```
// on the same line, but other than that I will let you decide the format.
```

```
// Pass the vector of CharUnit's as a constant reference to receive full credit.
```

```
void printVector (const vector <CharUnit> &inputVector)
```

```
{
```

```
    for (int i=0 ; i < inputVector.size(); i++)
```

```
    {
        cout << inputVector[i].character << " ";
```

```
        cout << inputVector[i].frequency << endl;
```

```
    }
```

```
}
```


// 5. Finish the implementation of the global function makeLetterHistogram.

```
void makeLetterHistogram(vector <CharUnit> & charVector)
{
    ifstream inputFile("input.txt", ios::in);
    char tempchar;
```

//6. First, finish the while loop CONDITION on the next line.

// It should read in a character from the file "input.txt"

```
while ( input file >> tempchar )
{
```

//7. Now write the code that searches the charVector and updates

//the frequency.

```
if ( tempchar != ' ' ) // if not a space
    charVector[tempchar - 97].frequency++;
```

```
} //end of while loop
```

```
} //end of function
```

Problem 4: (5 points)

I want to make sure that you know how to pass a variable to a function by REFERENCE (as opposed to by VALUE). Assume that we have a global function called `functionExample()`. We would like to pass to it an integer variable by reference. Which function call and function signature combinations below could accomplish this goal?

a.

function call:

`functionExample(&mainVar);`

✓

function signature:

`void functionExample(int * thisVar)`

b.

function call:

`functionExample(mainVar);`

✓

function signature:

`void functionExample(int & thisVar)`

c.

function call:

`functionExample(mainVar);`

X

function signature:

`void functionExample (int thisVar)`

d.

function call:

`functionExample(& mainVar);`

X

function signature:

`void functionExample (int & thisVar)`

e. a & b

✓

f. none of the above

Problem 5 (20 points): Please create a more efficient implementation of the following class definitions using public inheritance and virtual functions. I would like for you to create a base class called `JusticeLeagueCharacters()` *that maximizes code reuse for this program*. Please use the `private` access specifier in this base class for all data members. Also I would like for you to have a *pure* virtual function in `JusticeLeagueCharacters()` for the member function `act()`. When `act()` is called, it should print out the value of the `currentCaption` string and indicate who said it. To receive full credit, the `act()` member function must be changed to be a constant member function. You can assume that the member function `returnToJusticeLeagueHQ()` has the exact same implementation in `Batman`, `Superman`, and `WonderWoman`. It just prints “Here I go!” Here are the classes that you will need to *change* to incorporate into the new class hierarchy that you create!

class Batman

```
{ public:
    Batman(): numberOfItemsOnBatBelt(5), robinAlive(true), currentCaption("Pow!!"), weaponType(0),
    health(10) {}
    void act();
    void returnTo JusticeLeagueHQ();
    void talkDeepVoice();
private:
    int health;
    int weaponType;
    string currentCaption;
    int numberOfItemsOnBatBelt;
    bool robinAlive;};
```

class Superman

```
{ public:
    Superman(): leapingHeight(82.5), kryptonitePresent(false), currentCaption("Up up and away!!"),
    weaponType(0), health(10) {}
    void act();
    void returnTo JusticeLeagueHQ();
    void leapTallBuildings();
    void kickBatmansButt();
private:
    int health;
    int weaponType;
    string currentCaption;
    double leapingHeight;
    bool kryptonitePresent; };
```

class WonderWoman

```
{ public:
    WonderWoman(): invisibleJetCrashProbability(0.15),magicLassoStrength(5), currentCaption("Where
did I park my invisible jet?"), weaponType(0), health(10) {}
    void act();
    void returnTo JusticeLeagueHQ();
    void flyInvisibleJet();
    void fendOffBullets();
private:
    int health;
    int weaponType;
    string currentCaption;
    double invisibleJetCrashProbability;
    int magicLassoStrength; }
```

The following code in main should be able to run correctly with your NEW class hierarchy.

```
int main()
{
    Batman firstBM;
    WonderWoman firstWW;
    Superman firstSP;

    JusticeLeagueCharacters *BatDude = &firstBM;
    JusticeLeagueCharacters *SuperGirl = &firstWW;
    JusticeLeagueCharacters *SuperBoy = &firstSP;

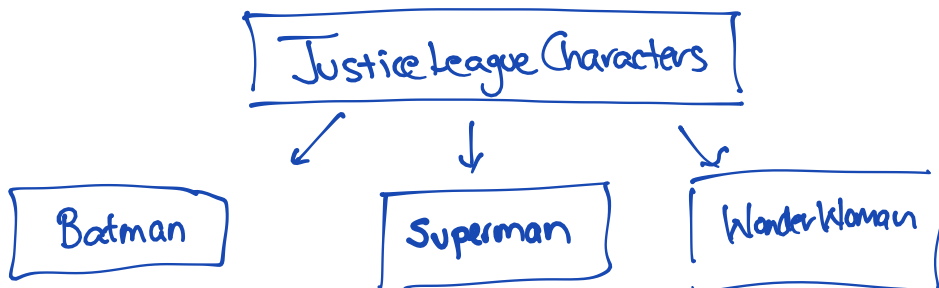
    BatDude->act();
    SuperGirl->act();
    SuperBoy->act();

}
```

The output from this main function should give:

Batman says: Pow!!
Wonderwoman says: Where did I park my invisible jet?
Superman says: Up up and away!!

(a) Graphically illustrate your new class hierarchy that incorporates the abstract class JusticeLeagueCharacters and concrete classes Batman, Superman, and WonderWoman.



(b) Show the class interface for JusticeLeagueCharacters here. For simplicity, only include set and get functions that you need in other parts of problem 5. Please show the implementation of the constructor inside the class interface using an initializer list.

```

class JusticeLeagueCharacters
{
    private:
        int health;
        int weaponType;
        string currentCaption;
    public:
        virtual void act() = 0;
        JusticeLeagueCharacters(string mess_): health(10), weaponType(0),
            currentCaption(mess_) {}
        void returnToJusticeLeagueHQ();
        string getCaption() { return currentCaption; }
};

```

(c) Show the class interface of a new Superman class that is incorporated in your class hierarchy. For simplicity DO NOT include any extra set or get functions. Please show the implementation of the constructor inside the class interface using an initializer list.

```

class Superman: public JusticeLeague
{
    public:
        Superman(): JusticeLeague("Up up and away!!"),
            leapingHeight(200.5), KryptonitePresent(false) {}

        void act() const;
        void leapTallBuildings();
        void KickBatmansButt();
    private:
        double leapingHeight;
        bool KryptonitePresent;
};

```

(d) Show all implementations of `act()` that you need for Batman, Superman, and Wonderwoman to ensure that when it is called that it prints the contents of `currentCaption`. Also in each function of `act()` that you create make sure you indicate who is speaking BEFORE you print out the `currentCaption` as seen in the output described above. Remember that I want `act()` to be designated as a constant member function.

```
void Wonderwoman::act() const
{
    cout << "Wonderwoman says:" << getCaption() << endl;
}
```

```
void Superman::act() const
{
    cout << "Superman says: " << getCaption() << endl;
}
```

```
void Batman::act() const
{
    cout << "Batman says: " << getCaption() << endl;
}
```

Problem 6 (5 points):

The program listed below is stored in a file that is called `problem6.cc` and to compile the program assume Dr. Davis types in the following on his computer.

```
g++ problem6.cc -o Caitlyn ✓
```

At the command prompt Dr. Davis types in the following:

Caitlyn Michael

What is the resulting output? Put your answer in the box below.

2
Caitlyn is cool!

```
#include <iostream>
#include <string>
using namespace std;

int main(int argc, char * argv[])
{
    cout << argc << endl;
    if (argc < 2)
        cout << "You need more arguments" << endl;
    else
        cout << argv[0] << " is cool!!" << endl;
}
```

Problem 7 (10 points):

Does the following code produce compile error(s)? If it does not, please show the output of the program. If it does, please show where the error(s) are and indicate WHY they occur.

```
#include <iostream>
using namespace std;
```

```
class Base
{
public:
    virtual void Func1()=0;
    void Func2();
};
```

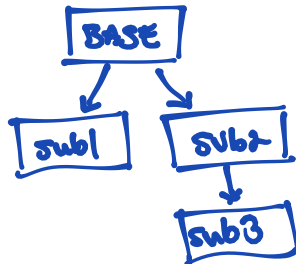
```
class Sub1 : public Base
{
public:
    void Func2();
};
```

```
class Sub2 : public Base
{
public:
    virtual void Func1();
    void Func2();
    void Func3();
};
```

```
class Sub3 : public Sub2
{
public:
    void Func2();
};
```

```
//----- Implementation
// Base
void Base::Func2()
{ std::cout << "Hello from Base::Func2()" << std::endl;
}
```

```
// Sub1
```



```
void Sub1::Func2()
{ std::cout << "Hello from Sub1::Func2()" << std::endl;
}
```

```
// Sub2
void Sub2::Func1()
{ std::cout << "Hello from Sub2::Func1()" << std::endl;
}
void Sub2::Func2()
{ std::cout << "Hello from Sub2::Func2()" << std::endl;
}
void Sub2::Func3()
{ std::cout << "Hello from Sub2::Func3()" << std::endl;
}
```

```
// Sub3
void Sub3::Func2()
{ std::cout << "Hello from Sub3::Func2()" << std::endl;
}
```

```
//----- Global Functions
```

```
void globalFunction(Base &b)
{
    b->Func1();
    b->Func2();
}
```

```
//----- Main
int main()
{
    Sub1 s1;
    Sub2 s2;
    Sub3 s3;
```

```
    globalFunction(s2);
    globalFunction(s3);
}
```

*b is a reference
not a pointer*

} → ERROR

Sub1 s1; → ERROR

*Sub1 is an
abstract class*

*because there
is no implementation
of Func1, which is
a pure virtual function.*

Problem 8 (10 points):

In class we discussed the following time class. I would like for you to show me an overload of the post-increment and pre-increment operator. If either operator is used on the time class I would like for you to increase the hour by 1. I would like for you to use military time. Remember that if the current hour is 23 and it is incremented, it should give an hour of 0. Make sure that the post- and pre-increment follow the same rules as we have discussed in class when they are used with fundamental types. ALSO INDICATE WHETHER THE POST OR THE PRE-INCREMENT WOULD BE FASTER ! WHY?

```
class Time
{
public:
    Time(int=0,int=0,int=0);
    //set functions
    void setTime(int,int,int);
    void setHour(int);
    void setMinute(int);
    void setSecond(int);
    //Insert function prototype for post and pre increment here.
    Time operator++(int); // post
    Time & operator++(); //pre
    //get functions
    int getHour() const;
    int getMinute() const;
    int getSecond() const;
    //general member functions
    void printTime() const;
private:
    int hour; //0-23 hours
    int minute; //0 to 59
    int second; //0 to 59
    const int lunchHour;
}; //end of time class
//-----
// Show implementation of pre-increment here
//-----
```

```
Time & Time::operator++()
{
    hour = (hour+1)%24;
    return (*this);
}
```

```
//-----
// Show implementation of post-increment here
//-----
```

```
Time Time::operator++(int)
{
    Time tempval(*this);
    hour = (hour+1)%24;
    return tempval;
}
```

Pre-increment is faster because there is less overhead (esp. no copy constructor).

Problem 9 (10 points): Pointer-Scoping-Mania

Please trace through this program and show the output. Please put a box around your final answer.

```
#include <iostream>
using namespace std;
```

```
int value = 10;
int * pointer = &value;
```

```
int cloudingFunction();
```

```
int main()
{
    int value = 100;
    int * pointer = &value;
    int * pointer2 = new int[10];
    int & value2 = value;
    int * & bigDaddy = pointer2;
```

```
    for (int i=0; i<10; i++)
    { pointer2[i] = 2*i; }

    cout << *pointer << endl;
```

```
    ::pointer = pointer2 + 5;
    pointer = pointer2 + 2;
    *pointer = cloudingFunction()+value2;
    *(pointer+1) = cloudingFunction();
```

```
    cout << *::pointer << endl;
    bigDaddy[3] = value2 + 42;
```

```
    for (int i=0; i<10; i++)
        if (pointer2+i != ::pointer)
            cout << *(pointer2+i) << endl;
}
```

```
int cloudingFunction()
{
    static int alphaPrime = 13;
```

```
    alphaPrime += 2;
```

```
    return(alphaPrime);
}
```

global
value
10

~~global
pointer~~

main value, value2
100

main
pointer

static in
alphaPrime

13
15
17

main
pointer

0
2
4
6
8
10
12
14
16
18

pointer2
big Daddy

142

::pointer

100
10
0
2
115
142
8
12
14
16
18

ORDER OF PRECEDENCE CHART

Precedence	Operator	Description	Associativity
1	::	Scope resolution	Left-to-right
2	++ --	Suffix/postfix increment and decrement	Left-to-right
	()	Function call	
	[]	Array subscripting	
	.	Element selection by reference	
	->	Element selection through pointer	
3	++ --	Prefix increment and decrement	Right-to-left
	+ -	Unary plus and minus	
	! ~	Logical NOT and bitwise NOT	
	(type)	Type cast	
	*	Indirection (dereference)	
	&	Address-of	
	sizeof	Size-of	
	new, new[]	Dynamic memory allocation	
	delete, delete[]	Dynamic memory deallocation	
4	.* ->*	Pointer to member	Left-to-right
5	* / %	Multiplication, division, and remainder	
6	+ -	Addition and subtraction	
7	<< >>	Bitwise left shift and right shift	
8	< <=	For relational operators < and ≤ respectively	
	> >=	For relational operators > and ≥ respectively	
9	== !=	For relational = and ≠ respectively	
10	&	Bitwise AND	
11	^	Bitwise XOR (exclusive or)	
12		Bitwise OR (inclusive or)	
13	&&	Logical AND	
14		Logical OR	
15	?:	Ternary conditional ^[1]	Right-to-left
	=	Direct assignment (provided by default for C++ classes)	
	+= -=	Assignment by sum and difference	
	*= /= %=	Assignment by product, quotient, and remainder	
	<<= >>=	Assignment by bitwise left shift and right shift	
16	&= ^= =	Assignment by bitwise AND, XOR, and OR	
	throw	Throw operator (for exceptions)	
17	,	Comma	Left-to-right